

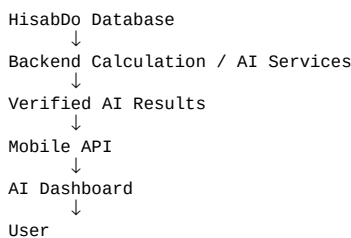
# HisabDo AI — Day 17

## Mobile AI Dashboard Implementation Specification

Scope: AI dashboard screens, API integration, UI states, recommendation detail, Ask HisabDo AI, responsiveness, and end-to-end testing.

### 1. Objective

The objective is to provide a complete mobile experience for HisabDo AI. The mobile application should consume verified backend AI results and present them in a clear dashboard. The mobile layer must not calculate Financial Health, spending percentages, forecasts, budgets, recommendation priority, or severity. Backend services remain the source of truth.



### 1. Create/prepare AI dashboard screen

Create a central AI Dashboard containing Financial Health, Spending Patterns, Anomalies, Cash-Flow Forecast, Budget Recommendations, AI Recommendations, and an entry point for Ask HisabDo AI. Cards should be independent so one unavailable feature does not block the complete dashboard.

### 2. Integrate Financial Health API

Connect the mobile dashboard to the authenticated backend endpoint, for example GET `/api/mobile/ai/dashboard` or a dedicated Financial Health endpoint. The authenticated user identity must come from the backend authentication context rather than a user ID supplied by the client.

### 3. Display Financial Health Score

Show the verified 0–100 score, classification, and optional factor/explanation information. Do not recalculate the score on mobile. Invalid or unverified values should not be displayed as trusted results.

### 4. Display Spending Patterns

Show useful category-level patterns such as current spending, previous spending, percentage change and trend. The UI should use backend values and explanations rather than recalculating percentages locally.

### 5. Display Anomalies

Display verified anomaly type, severity, reason and optional amount. A user should be able to understand why an item was flagged without the UI claiming unsupported causes such as fraud.

### 6. Display Cash-Flow Forecast

Show forecast income, forecast expense, expected saving, confidence and closing balance only when the closing balance is based on a trusted opening balance.

### 7. Display Budget Recommendations

Show category, historical average, recommended budget, existing budget when available, variance, reason and data sufficiency. The mobile client should not generate a new budget.

## **8. Display AI Recommendations**

Show recommendation title/type, short explanation, action, priority and severity where available. Expected benefit should only appear when verified by backend data.

## **9. Display Priority and Severity**

Priority and severity are backend recommendation outputs. Mobile should display them exactly as returned and must not recalculate or modify them.

## **10. Add recommendation detail view**

Selecting a recommendation opens a detail screen with Why, Financial Pattern, Priority, Severity if available, Action, Expected Benefit if verified, and Limitation when required.

## **11. Add Ask HisabDo AI interface**

Provide a question input and answer area. The flow is User Question → POST /api/mobile/ai/ask → Intent Detection → Verified Data Retrieval → AI Explanation → Verified Answer → Mobile.

## **12. Add loading states**

Use loading indicators or skeleton cards while dashboard data is being retrieved. Prevent confusing blank screens and avoid showing stale partial data as a successful response.

## **13. Add empty states**

When no verified insights are available, show a clear message such as 'No AI financial insights are available yet.' Each feature may also have its own empty state.

## **14. Add API error states**

Display a user-friendly error message when an API request fails. Avoid exposing stack traces, internal exception messages, database details, or sensitive information.

## **15. Add retry functionality**

Provide Retry on retryable failures. Retry should call the API again and return the UI to Loading before Success, Empty, or Error.

## **16. Validate mobile responsiveness/UI**

Validate different screen widths, text wrapping, long category names, long AI explanations, cards, scrolling, orientation, touch targets, loading states, error messages, and accessibility/readability.

## **17. Perform end-to-end mobile testing**

Test the complete flow from authenticated mobile user through backend APIs to verified AI output and final UI. Validate success, empty, error, retry, recommendation detail and Ask HisabDo AI scenarios.

## 2. Recommended AI Dashboard Layout

```

■ HisabDo AI ■
■ Your Financial Intelligence ■
■ Financial Health ■
■ Score: 72/100 Good ■
■ Spending Patterns ■
■ Food ↑ Transport ↓ Bills → ■
■ Anomalies ■
■ 2 items require review ■
■ Cash-Flow Forecast ■
■ Income | Expense | Saving ■
■ Budget Recommendations ■
■ Category | Recommended Budget ■
■ AI Recommendations ■
■ • Review spending [Medium] ■
■ • Watch cash flow [High] ■
■ Ask HisabDo AI [Ask a question] ■

```

### 3. API Contract

| Endpoint                                | Purpose                                                            |
|-----------------------------------------|--------------------------------------------------------------------|
| GET /api/mobile/ai/dashboard            | Load dashboard AI data in one authenticated response.              |
| POST /api/mobile/ai/ask                 | Submit a natural-language financial question.                      |
| GET /api/mobile/ai/recommendations/{id} | Optional detail endpoint if recommendations are loaded separately. |

## 4. Dashboard Response Model

```
DashboardResponse
  FinancialHealth
  SpendingPatterns[]
  Anomalies[]
  CashFlowForecast
  BudgetRecommendations[]
  Recommendations[]
```

The response is intentionally modular. A feature can be null or empty without preventing the remaining cards from being displayed. The mobile client should render available verified sections and show a feature-level empty state where data is unavailable.

## 5. Loading, Empty, Error and Retry States

| State   | Mobile behavior                                                     |
|---------|---------------------------------------------------------------------|
| Loading | Show skeleton/card loader and disable duplicate request actions.    |
| Success | Render all available verified data.                                 |
| Empty   | Show helpful message and explain what the user can do next.         |
| Error   | Show safe error message and Retry when retryable.                   |
| Retry   | Return to Loading, call API again, then render Success/Empty/Error. |

## 6. Recommendation Detail Screen

- Recommendation title/type
- Short AI summary
- Why the recommendation was generated

- Verified financial pattern
- Priority
- Severity when available
- Actionable suggestion
- Expected benefit only when verified
- Limitation when required

## 7. Ask HisabDo AI Flow

User types question  
↓  
POST /api/mobile/ai/ask  
↓  
Authenticated User  
↓  
Intent Detection  
↓  
Verified HisabDo Data  
↓  
AI Explanation  
↓  
Verified Answer + Limitation  
↓  
Mobile Answer Screen

## 8. Responsive UI Validation

- Test small, medium and large phone widths.
- Check portrait and landscape where supported.
- Verify long AI text wraps without clipping.
- Verify recommendation cards do not overflow horizontally.
- Verify buttons and retry actions have usable touch areas.
- Verify empty/error states remain readable.
- Verify numeric values remain legible.
- Verify scrolling works when multiple cards are present.
- Verify accessibility labels and contrast/readability according to the mobile framework guidelines.
- Verify slow-network behavior and loading transitions.

## 9. End-to-End Test Matrix

| Scenario                              | Expected result                                                                       |
|---------------------------------------|---------------------------------------------------------------------------------------|
| Valid authenticated dashboard request | All available verified cards render.                                                  |
| No AI data                            | Dashboard shows safe empty state.                                                     |
| Financial Health available only       | Health card renders; other cards show independent empty states.                       |
| API timeout                           | Safe error + Retry.                                                                   |
| Retry succeeds                        | Loading → Success.                                                                    |
| Invalid Health score                  | Response rejected/flagged; invalid score not trusted.                                 |
| Recommendation with Priority/Severity | Values display exactly from backend.                                                  |
| Recommendation without Severity       | Severity section is hidden, not invented.                                             |
| Expected benefit missing              | Benefit section is hidden.                                                            |
| Recommendation detail opened          | Detail page shows why, pattern, priority, severity, action and applicable limitation. |
| Ask HisabDo valid question            | Loading → verified answer.                                                            |
| Ask HisabDo empty question            | Validation error; no API request required.                                            |
| Ask API failure                       | Safe error + Retry.                                                                   |
| Different authenticated user          | Only that user's verified data is returned.                                           |
| Long explanation                      | Text wraps correctly without layout break.                                            |
| Repeated dashboard refresh            | No duplicate/incorrect data shown.                                                    |

## 10. Security and Data Integrity

- Use authenticated API requests.
- Derive UserId from the authenticated backend context.
- Do not trust a client-supplied UserId.
- Do not allow mobile to modify financial source values.
- Do not allow mobile to modify recommendation priority/severity.
- Return only data authorized for the authenticated user.
- Do not expose internal exception details.

## 11. .NET 8 Reference Implementation

The companion C# file contains DTOs, authenticated ASP.NET Core endpoints, mobile loading states, dashboard ViewModel/state coordination, Ask HisabDo flow, retry handling, empty-state detection, API error handling, and basic UI validation helpers.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace HisabDo.Mobile.AI;

// .NET 8 / ASP.NET Core reference API + mobile-facing state models.
// The mobile client should use the API response as the source of truth.
// It must not calculate financial scores, forecasts, recommendations,
// priority, or severity locally.

public sealed record FinancialHealthDto(
    bool IsVerified,
    decimal Score,
    string Classification,
    IReadOnlyDictionary<string, decimal> Factors,
    string? Explanation);

public sealed record SpendingPatternDto(
```

```

    bool IsVerified,
    string Category,
    decimal CurrentAmount,
    decimal PreviousAmount,
    decimal PercentageChange,
    string Trend,
    string? Explanation);

public sealed record AnomalyDto(
    bool IsVerified,
    string Type,
    decimal? Amount,
    string Severity,
    string Reason,
    string? Explanation);

public sealed record CashFlowForecastDto(
    bool IsVerified,
    decimal ForecastIncome,
    decimal ForecastExpense,
    decimal ExpectedSaving,
    decimal? ExpectedClosingBalance,
    string Confidence,
    string? Explanation);

public sealed record BudgetRecommendationDto(
    bool IsVerified,
    string Category,
    decimal HistoricalAverage,
    decimal RecommendedBudget,
    decimal? ExistingBudget,
    decimal? BudgetVariance,
    string Reason,
    string DataSufficiency,
    string? Explanation);

public sealed record RecommendationDto(
    string RecommendationId,
    bool IsVerified,
    string Type,
    string Priority,
    string? Severity,
    string Reason,
    string Action,
    decimal? VerifiedAmount,
    decimal? VerifiedPercentage,
    string? VerifiedExpectedBenefit,
    string? Limitation);

public sealed record DashboardResponse(
    FinancialHealthDto? FinancialHealth,
    IReadOnlyList<SpendingPatternDto> SpendingPatterns,
    IReadOnlyList<AnomalyDto> Anomalies,
    CashFlowForecastDto? CashFlowForecast,
    IReadOnlyList<BudgetRecommendationDto> BudgetRecommendations,
    IReadOnlyList<RecommendationDto> Recommendations);

public sealed record AskHisabDoRequest(string Question);

public sealed record AskHisabDoResponse(
    bool IsVerified,
    string Answer,
    string? Intent,
    string? Limitation);

public sealed record ApiError(
    string Code,
    string Message,
    bool Retryable);

public enum MobileLoadState
{
    Idle,
    Loading,
    Success,
    Empty,
    Error
}

public sealed record MobileScreenState<T>(
    MobileLoadState State,
    T? Data,
    string? Message,
    bool CanRetry)
{
    public static MobileScreenState<T> Loading() =>
        new(MobileLoadState.Loading, default, null, false);

    public static MobileScreenState<T> Success(T data) =>
        new(MobileLoadState.Success, data, null, false);

    public static MobileScreenState<T> Empty(string message) =>
        new(MobileLoadState.Empty, default, message, true);

    public static MobileScreenState<T> Error(string message, bool canRetry = true) =>

```

```

        new(MobileLoadState.Error, default, message, canRetry);
    }

    public interface ICurrentUser
    {
        string UserId { get; }
    }

    public interface IMobileAiDashboardService
    {
        Task<DashboardResponse> GetDashboardAsync(
            string authenticatedUserId,
            CancellationToken cancellationToken);

        Task<AskHisabDoResponse> AskAsync(
            string authenticatedUserId,
            string question,
            CancellationToken cancellationToken);
    }

    [ApiController]
    [Authorize]
    [Route("api/mobile/ai")]
    public sealed class AiDashboardController : ControllerBase
    {
        private readonly IMobileAiDashboardService _service;
        private readonly ICurrentUser _currentUser;

        public AiDashboardController(
            IMobileAiDashboardService service,
            ICurrentUser currentUser)
        {
            _service = service;
            _currentUser = currentUser;
        }

        [HttpGet("dashboard")]
        public async Task<ActionResult<DashboardResponse>> GetDashboard(
            CancellationToken cancellationToken)
        {
            var userId = _currentUser.UserId;

            if (string.IsNullOrEmpty(userId))
                return Unauthorized(new ApiError(
                    "AUTH_REQUIRED",
                    "Authenticated user context is required.",
                    false));

            try
            {
                var dashboard = await _service.GetDashboardAsync(
                    userId, cancellationToken);

                return Ok(dashboard);
            }
            catch (OperationCanceledException)
            {
                return StatusCode(499, new ApiError(
                    "REQUEST_CANCELLED",
                    "The request was cancelled.",
                    true));
            }
            catch (Exception)
            {
                return StatusCode(500, new ApiError(
                    "DASHBOARD_ERROR",
                    "Unable to load AI dashboard.",
                    true));
            }
        }

        [HttpPost("ask")]
        public async Task<ActionResult<AskHisabDoResponse>> Ask(
            [FromBody] AskHisabDoRequest request,
            CancellationToken cancellationToken)
        {
            var userId = _currentUser.UserId;

            if (string.IsNullOrEmpty(userId))
                return Unauthorized(new ApiError(
                    "AUTH_REQUIRED",
                    "Authenticated user context is required.",
                    false));

            if (request is null || string.IsNullOrEmpty(request.Question))
                return BadRequest(new ApiError(
                    "QUESTION_REQUIRED",
                    "Please enter a financial question.",
                    false));

            try
            {
                var answer = await _service.AskAsync(
                    userId,
                    request.Question.Trim(),

```

```

        cancellationToken);
    }
    return Ok(answer);
}
catch (OperationCanceledException)
{
    return StatusCode(499, new ApiError(
        "REQUEST_CANCELLED",
        "The request was cancelled.",
        true));
}
catch (Exception)
{
    return StatusCode(500, new ApiError(
        "ASK_ERROR",
        "Unable to process the question.",
        true));
}
}
}

// Example mobile ViewModel/state coordinator.
// This is framework-neutral C# and can be adapted to .NET MAUI,
// Xamarin migration code, React Native bridge code, or another client.
public sealed class AiDashboardViewModel
{
    private readonly Func<CancellationToken, Task<DashboardResponse>> _loadApi;
    private readonly Func<string, CancellationToken, Task<AskHisabDoResponse>> _askApi;

    public MobileScreenState<DashboardResponse> Dashboard { get; private set; }
    = new(MobileLoadState.Idle, null, null, false);

    public MobileScreenState<AskHisabDoResponse> AskState { get; private set; }
    = new(MobileLoadState.Idle, null, null, false);

    public AiDashboardViewModel(
        Func<CancellationToken, Task<DashboardResponse>> loadApi,
        Func<string, CancellationToken, Task<AskHisabDoResponse>> askApi)
    {
        _loadApi = loadApi;
        _askApi = askApi;
    }

    public async Task LoadDashboardAsync(CancellationToken cancellationToken = default)
    {
        Dashboard = MobileScreenState<DashboardResponse>.Loading();

        try
        {
            var data = await _loadApi(cancellationToken);

            if (IsDashboardEmpty(data))
            {
                Dashboard = MobileScreenState<DashboardResponse>.Empty(
                    "No AI financial insights are available yet.");
                return;
            }

            Dashboard = MobileScreenState<DashboardResponse>.Success(data);
        }
        catch (OperationCanceledException)
        {
            Dashboard = MobileScreenState<DashboardResponse>.Error(
                "Loading was cancelled.", true);
        }
        catch (Exception)
        {
            Dashboard = MobileScreenState<DashboardResponse>.Error(
                "Unable to load AI dashboard. Please try again.", true);
        }
    }

    public Task RetryDashboardAsync(CancellationToken cancellationToken = default) =>
        LoadDashboardAsync(cancellationToken);

    public async Task AskHisabDoAsync(
        string question,
        CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrEmpty(question))
        {
            AskState = MobileScreenState<AskHisabDoResponse>.Error(
                "Please enter a question.", false);
            return;
        }

        AskState = MobileScreenState<AskHisabDoResponse>.Loading();

        try
        {
            var answer = await _askApi(
                question.Trim(), cancellationToken);

            if (answer is null ||
                string.IsNullOrEmpty(answer.Answer))

```

```

        {
            AskState = MobileScreenState<AskHisabDoResponse>.Empty(
                "No answer is available for this question.");
            return;
        }

        AskState = MobileScreenState<AskHisabDoResponse>.Success(answer);
    }
    catch (Exception)
    {
        AskState = MobileScreenState<AskHisabDoResponse>.Error(
            "Unable to get an answer. Please try again.", true);
    }
}

private static bool IsDashboardEmpty(DashboardResponse data)
{
    return data is null
        || (data.FinancialHealth is null
            && data.SpendingPatterns.Count == 0
            && data.Anomalies.Count == 0
            && data.CashFlowForecast is null
            && data.BudgetRecommendations.Count == 0
            && data.Recommendations.Count == 0);
}

// UI rendering contract.
// Every screen should map to one of these states:
// Loading → Success / Empty / Error.
public static class MobileUiContract
{
    public static string RenderDashboardState(
        MobileScreenState<DashboardResponse> state) =>
        state.State switch
        {
            MobileLoadState.Loading => "Show skeleton/loading indicators.",
            MobileLoadState.Success => "Render all available AI cards.",
            MobileLoadState.Empty => "Show empty-state guidance.",
            MobileLoadState.Error => "Show error and Retry action.",
            _ => "Idle."
        };
}

// UI validation helpers.
public static class MobileUiValidator
{
    public static void ValidateDashboard(DashboardResponse data)
    {
        if (data is null)
            throw new InvalidOperationException("Dashboard response is null.");

        if (data.FinancialHealth is { } health)
        {
            if (health.IsVerified && (health.Score < 0 || health.Score > 100))
                throw new InvalidOperationException(
                    "Financial Health score must be between 0 and 100.");
        }

        foreach (var recommendation in data.Recommendations)
        {
            if (recommendation.IsVerified)
            {
                if (string.IsNullOrEmpty(recommendation.Type))
                    throw new InvalidOperationException(
                        "Verified recommendation type is required.");

                if (string.IsNullOrEmpty(recommendation.Priority))
                    throw new InvalidOperationException(
                        "Verified recommendation priority is required.");
            }
        }
    }
}

```

/\*  
Recommended mobile screen structure:

#### AI Dashboard

- Financial Health Card
- Spending Patterns Card
- Anomalies Card
- Cash-Flow Forecast Card
- Budget Recommendations Card
- AI Recommendations Card
- Ask HisabDo AI Entry

#### Recommendation Detail

- Title
- Why
- Financial Pattern
- Priority
- Severity (if available)
- Action
- Expected Benefit (only if verified)

■■■ Limitation (if required)

Every API screen:

Loading → Success / Empty / Error → Retry

\*/

## 12. Acceptance Checklist

- AI Dashboard screen created.
- Financial Health API integrated.
- Financial Health Score displayed from backend.
- Spending Patterns displayed.
- Anomalies displayed.
- Cash-Flow Forecast displayed.
- Budget Recommendations displayed.
- AI Recommendations displayed.
- Priority and Severity displayed from backend.
- Recommendation Detail view implemented.
- Ask HisabDo AI interface implemented.
- Loading states implemented.
- Empty states implemented.
- API error states implemented.
- Retry functionality implemented.
- Responsive UI validated.
- End-to-end mobile testing completed.
- Authenticated user isolation validated.
- Mobile does not calculate financial source-of-truth values.